

Conclusion

1. 实验一独立

Lab 1 (MapReduce) 是独立实现的，与后面的 Raft 共识算法无关。

2. 实验二、三、四递进

- **Lab 2:** 构建 Raft 日志共识层（主要是日志复制）

Lab 2A: 领导者选举 (Leader Election)

Lab 2B: 日志复制 (Log Replication) ← 你提到的主要部分

Lab 2C: 持久化 (Persistence) ← 节点重启后恢复状态

Lab 2D: 日志压缩 (Log Compaction with Snapshot) ← 防止日志无限增长

- **Lab 3:** 在 Raft 上构建单组 KV 服务
- **Lab 4:** 扩展到多组分片集群

3. 实验二的定位

日志层的共识算法：保证多副本日志一致性，解决"谁说了算"和"日志怎么同步"的问题。

4. 实验三的定位

状态机 + RAFT 一对一关系：

- 每个 Raft 实例对应一个 KVServer
- KVServer 从 Raft 的 `applyCh` 接收已提交的日志条目
- 应用到自己的内存 Map (状态机) 中

kvserver.go

```
1  type KVServer struct {  
2      rf          *raft.Raft          // 一对一关联的 Raft 节点  
3      applyCh      chan raft.ApplyMsg    // 接收 Raft 提交的日志  
4      stateMachine map[string]string    // 状态机：实际存储数据  
5  }
```

5. 实验四的核心概念

- **组 (Group):** 一个 Raft 集群（通常 3 或 5 个节点），内部运行 shardKV 的完整逻辑。

- **配置 (Config):** `shard -> gid` 的映射表。配置号从 0 递增，每次配置变更 (Join/Leave/Move) 都产生新配置。
- **分片 (Shard):** 数据分片单位，每个 Key 通过 `hash(key) % NShards` 决定所属 Shard。
- **数据迁移:** 配置变更时，Shard 所有权在 Group 间转移。

6. shardctrler 的定位

集群的大脑/控制平面:

- 维护配置历史
- 处理 Join/Leave/Move/Query 请求
- 执行负载均衡算法
- 本身也是容错服务 (内嵌 Raft)

7. shardkv 的定位

分片 KV 数据库:

- 每个 Group 内部是多个 Machine (Machine = Raft + shardKV)
- 额外增加了跨 Group 的配置同步、分片迁移、垃圾回收逻辑